encamina

PIENSA EN COLORES

Una consultora tecnológica que piensa en colores

*Para organizaciones vivas*

Microsoft 2025 Partner of the Year

Winner

Microsoft Solutions Partner
Digital & App Innovation
Azure

Microsoft Solutions Partner
Data & AI
Azure

Microsoft Solutions Partner
Infrastructure
Azure

Microsoft Solutions Partner
Modern Work

Microsoft Solutions Partner
Business Applications

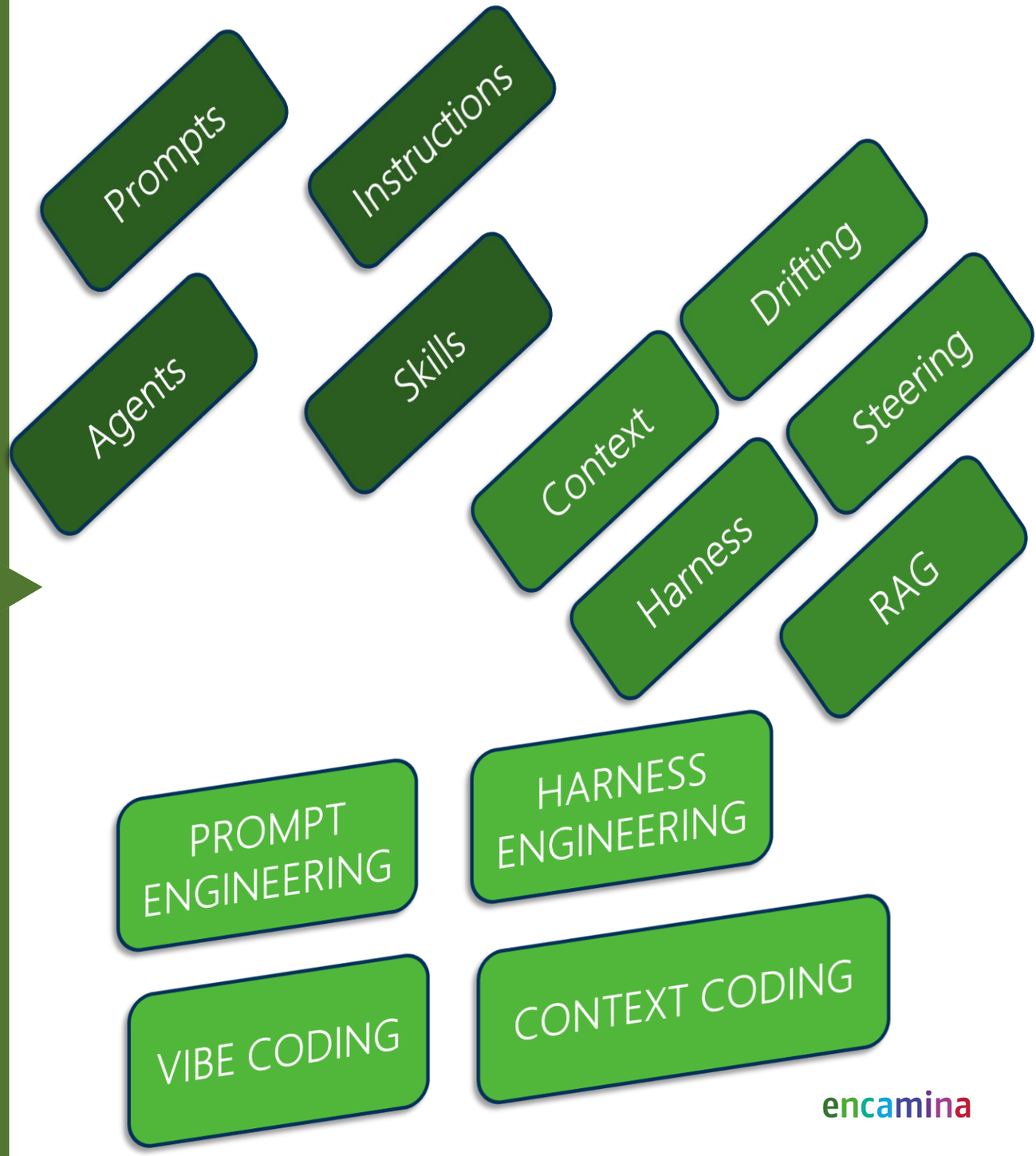
Microsoft Solutions Partner
Security



# Políticas y topologías de prompts para GitHub Copilot

Openfinance

# Antes de Empezar …
# ¿Qué nos suena?






# ¿Qué es un prompt?

Es la instrucción o solicitud que una persona proporciona a una Inteligencia Artificial (IA) para que genere una respuesta útil, como código, explicaciones, documentación o sugerencias.

Es la forma con la cual "conversamos" con una IA, por ejemplo, GitHub Copilot, para que nos ayude en tareas concretas.

La calidad del *prompt* afecta directamente a la calidad del resultado. Un *prompt* claro, específico y bien estructurado permite obtener resultados más útiles, precisos y seguros.




# ¿Qué debería incluir un prompt?

Contexto:

- *¿Qué estás intentando hacer?*
- *¿Qué parte del código estás trabajando?*

Objetivo claro, preciso:

- *¿Qué esperas que genere?*
- *¿Una función, una explicación, una prueba?*

Restricciones o formato:

- *¿Debe seguir un estilo específico?*
- *¿Debe usar un lenguaje de programación?*

# PROMPT ENGINEERING

La ingeniería de prompts es el arte de crear instrucciones que guían al modelo de IA para producir el resultado deseado.

Los prompts funcionan como la interfaz entre el usuario y las capacidades del modelo de IA. La calidad del resultado del modelo depende en gran medida, y muchas veces exclusivamente, de la calidad del prompt.

Los modelos de IA generativa son potentes, pero no conocen de forma inherente la intención de la persona. No pueden leer la mente, sino que hacen una interpretación basada en patrones aprendidos de sus datos.

# PROMPT ENGINEERING

Los *prompts* **mal redactados** pueden provocar:

- Resultados irrelevantes que no responden a las necesidades del usuario.
- Información inexacta, con errores u omisiones.
- Pérdida de tiempo. Se dedica más esfuerzo a refinar resultados en lugar de actuar sobre los conocimientos obtenidos.

Por otro lado, los *prompts* **bien elaborados** pueden lograr:

- Mejorar la eficiencia reduciendo la necesidad de múltiples revisiones.
- Generar resultados alineados con los objetivos del negocio.
- Menos riesgos al minimizar la posibilidad de interpretaciones erróneas.

# PROMPT ENGINEERING

## COMPONENTES DE UN PROMPT

### Instrucción

Es la parte que define la tarea y el objetivo para el modelo de IA. Debe ser clara y específica para que el modelo sepa exactamente qué espera el usuario.

Por ejemplo, si el usuario quiere que el modelo escriba un resumen de un artículo, debe incluir el artículo, la longitud deseada del resumen y los puntos principales que deben aparecer.

### Contexto

Es la parte que proporciona información adicional para orientar la salida del modelo.

Por ejemplo: la audiencia objetivo, el tono que usar, uso del resultado (profesional o personal), o el nivel de detalle.

El contexto debe ser relevante y adecuado para que el modelo pueda adaptar su respuesta a las necesidades del usuario.

# HARNESS ENGINEERING

## ¿Qué es un Harness en IA?

En el desarrollo con agentes de IA, un ***harness*** es el conjunto de componentes que rodean al modelo y permiten controlar cómo trabaja, qué herramientas puede utilizar, cómo accede al contexto y cómo ejecuta tareas.

El concepto ha ganado visibilidad con el desarrollo de agentes capaces de trabajar durante más tiempo y realizar tareas complejas de forma autónoma.

# HARNESS ENGINEERING

La cuestión ya no es únicamente qué modelo utilizamos o qué prompt escribimos, sino también cómo diseñamos el entorno para que el agente trabaje de forma fiable.

Podemos verlo como el entorno de ejecución y control de un agente.

El objetivo es reducir errores y mejorar la fiabilidad del desarrollo asistido por IA.

# HARNESS ENGINEERING

Por ejemplo, en un agente de programación, el *harness* puede encargarse de:

- Leer y modificar archivos del repositorio.
- Ejecutar comandos y pruebas.
- Gestionar el contexto y las instrucciones.
- Controlar los ciclos de ejecución del agente.
- Aplicar permisos, restricciones y verificaciones.

# HARNESS ENGINEERING

Por ejemplo, para una arquitectura con .NET 10, React, GitLab, Jenkins y Terraform, un ***harness*** podría integrar:

- Las instrucciones y convenciones del repositorio.
- El acceso controlado al código.
- Las herramientas de compilación y pruebas.
- Las validaciones de seguridad y calidad.
- Los mecanismos para revisar los cambios antes de integrarlos.

# Vibe Coding



*"Hay un nuevo tipo de programación que llamo «**vibe coding**», donde te entregas completamente a las vibras, abrazas los exponenciales y te olvidas de que el código siquiera existe. [...] Estoy construyendo un proyecto o una aplicación web, pero en realidad no estoy programando: solo veo cosas, digo cosas, ejecuto cosas y copio-pego cosas, y en su mayoría funciona."*

**Andrej Karpathy**

Founding team @ OpenAI
Former Director of AI @ Tesla

***3 de febrero de 2025***



https://x.com/karpathy/status/1886192184808149383

# Vibe Coding



Es la práctica de programación dependiente de forma directa de aplicar la Inteligencia Artificial en el flujo de trabajo (diario), mediante la cual cualquier persona (sea o no técnica) interactúa con herramientas de IA para, mediante prompts, dar instrucciones, describir y construir su propio software o configurar aplicaciones en plataformas como la nube.

Desde aplicaciones de escritorio o web hasta apps móviles, pasando por configurar plataformas como Kubernetes o Cloud Flare, a cualquier cosa que pueda escribirse con código o ejecutarse mediante *scripts* puede lograrse con «*vibe coding*»…

…en teoría.

# Context Coding



Es la evolución del «*vibe coding*», siendo una forma madura de realizarlo producto de la experiencia, la práctica continua y la adopción estricta de formas de trabajar, la adecuación de topologías de prompts y la implantación institucionalizada de políticas sobre los mismos.

Combina normas organizacionales, conjunto de prompts reutilizables (y constantemente mantenidos), conocimientos sobre fortalezas, debilidades y restricciones de los modelos de IA disponibles, junto a herramientas que enriquezcan las capacidades inherentes de dichos modelos, según se necesiten para diversas tareas.

# Context Coding



Post

**Andrej Karpathy** ✓
@karpathy

+1 for "context engineering" over "prompt engineering".

People associate prompts with short task descriptions you'd give an LLM in your day-to-day use. When in every industrial-strength LLM app, context engineering is the delicate art and science of filling the context window with just the right information for the next step. Science because doing this right involves task descriptions and explanations, few shot examples, RAG, related (possibly multimodal) data, tools, state and history, compacting... Too little or of the wrong form and the LLM doesn't have the right context for optimal performance. Too much or too irrelevant and the LLM costs might go up and performance might come down. Doing this well is highly non-trivial. And art because of the guiding intuition around LLM psychology of people spirits.

On top of context engineering itself, an LLM app has to:
- break up problems just right into control flows
- pack the context windows just right
- dispatch calls to LLMs of the right kind and capability
- handle generation-verification UIUX flows
- a lot more - guardrails, security, evals, parallelism, prefetching, ...

So context engineering is just one small piece of an emerging thick layer of non-trivial software that coordinates individual LLM calls (and a lot more) into full LLM apps. The term "ChatGPT wrapper" is tired and really, really wrong.

> **tobi lutke** ✓ @tobi · Jun 19, 2025
> I really like the term “context engineering” over prompt engineering.
>
> It describes the core skill better: the art of providing all the context for the task to be plausibly solvable by the LLM.

https://x.com/karpathy/status/1937902205765607626

# De Vibe Coding a Context Coding



## 1. Vibe Coding: «Dime qué quieres y lo construimos»

Se describe una funcionalidad en lenguaje natural.

La IA genera código y propone cambios.

Se prioriza la rapidez y obtener un resultado visible.

El usuario puede no comprender cómo funciona internamente.

Riesgo: conseguir una aplicación que parece funcionar, pero que tiene vulnerabilidades, errores de diseño o problemas operativos.

# De Vibe Coding a Context Coding



## 2. Context Coding: «Antes de programar, entiende el sistema»

La IA recibe información relevante sobre el proyecto:
Arquitectura y estructura del repositorio.

Reglas de negocio y requisitos funcionales.
Convenciones de programación y patrones existentes.

Requisitos de seguridad, permisos y protección de datos.
Pruebas, restricciones de infraestructura y criterios de aceptación.

Objetivo: generar cambios coherentes con el sistema real, no simplemente código que parezca correcto de forma aislada.

# De Vibe Coding a Context Coding



3. Ingeniería con agentes y controles: «Además de entender, debe verificarse»

Se incorporan mecanismos para que la IA trabaje dentro de límites definidos:

Ejecución automática de pruebas.
Análisis estático y comprobaciones de seguridad.
Permisos restringidos para acceder a archivos, secretos e infraestructura.
Revisión de cambios antes de fusionar código.
Despliegues controlados y posibilidad de recuperación.

Objetivo: que la velocidad de la IA no comprometa la seguridad ni la calidad del producto.

# De Vibe Coding a Context Coding

Imaginemos que queremos desarrollar una API en .NET 10, un frontend React y aprovisionar recursos en Azure mediante Terraform.



| Vibe Coding |
| --- |
| «Créame una API y despliega la en AWS». |
| Genera recursos y permisos para que funcione. |
| Prueba que la aplicación responde. |
| Puede desplegar cambios directamente. |

| Context Coding |
| --- |
| «Crea la API respetando nuestra arquitectura, convenciones, modelo de identidad y requisitos de seguridad». |
| Aplica los principios de mínimo privilegio y las restricciones de red definidas. |
| Añade pruebas funcionales, validaciones de seguridad y comprobaciones de infraestructura. |
| Exige revisiones y aprobación antes de modificar producción. |




# Topologías de Prompts

Cada modelo y cada asistente de IA tiene sus particularidades a la hora de trabajar.

Cada proveedor tecnológico propone o impone una serie de condiciones y formas de trabajar con sus modelos y asistentes que se deben adaptar a la idiosincrasia de la empresa que adopta esas tecnologías.

**Adoptar implica Adaptar**



Para el caso de GitHub Copilot (GHC), estas topologías se refieren a los diferentes tipos de archivos y formatos que podemos usar como ingeniería de prompts para facilitar o promover una forma de trabajar orientada al «*context coding*» a nivel profesional y empresarial.

A día de hoy, se distinguen los siguientes tipos de archivos:

- Prompts
- Instrucciones
- Agentes
- Habilidades

En GHC, por ahora, se emplean seis topologías que se pueden usar para proporcionar contexto e instrucciones, las cuales deben estar (habitualmente) en el directorio `.github` o a nivel raíz del proyecto.



## Evolución de los enfoques de IA

### Prompts (Enfoque tradicional)

La capa más antigua y con menor versatilidad.



**0%**

### Instrucciones (Reglas del juego)

Archivos base compartidos por la mayoría de LLMs.



**100%**

### Agentes (Especialización avanzada)

Capas de IA personalizadas para interactuar con el entorno.



**100%**

### Habilidades (Capacidades técnicas)

Funciones y herramientas específicas que el agente puede ejecutar.



**100%**

Los enfoques de IA han evolucionado desde prompts simples hasta agentes especializados con habilidades técnicas.



# 1. Instrucciones del repositorio para GitHub Copilot

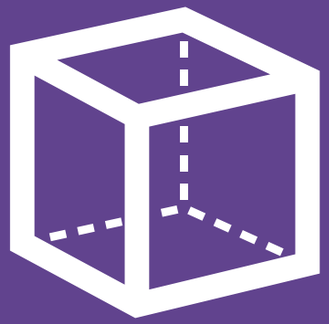
Especifica instrucciones y preferencias a nivel de repositorio, **en un solo archivo**, que se aplican a cualquier conversación (*chat*) realizada en el contexto del repositorio.

El archivo se debe llamar **siempre** `copilot-instructions.md`, el cual es detectado automáticamente, aplicando las mismas instrucciones a todas las solicitudes que se realicen.

Se usa para definir:

- Estilo de codificación y convenciones **particulares, propias** o *ad-hoc* del proyecto.
- Declaraciones sobre la pila (*stack*) tecnológica y librerías preferidas.
- Patrones arquitectónicos **particulares** que se deben seguir o evitar.
- Requisitos de seguridad y enfoques de manejo de errores.
- Estándares de documentación **propios** del proyecto.





# DEMO 1

encamina



## 2. Instrucciones específicas o personalizadas

Permiten guardar instrucciones comunes y contexto relevante que luego se puede reutilizar en los mensajes del chat. Son archivos con la extensión `.instructions.md`.

Definen directrices y reglas comunes que influyen automáticamente en cómo la IA genera código y gestiona otras tareas de desarrollo. En lugar de incluir manualmente el contexto en cada mensaje del chat, especifica instrucciones personalizadas en un archivo Markdown para garantizar respuestas coherentes de la IA que se alineen con las prácticas de codificación y los requisitos del proyecto o la organización.

Se pueden configurar instrucciones personalizadas para que se apliquen automáticamente a todas las solicitudes de chat o solo a archivos específicos.

Se recomienda su utilización para indicar el «qué» más que el «cómo».

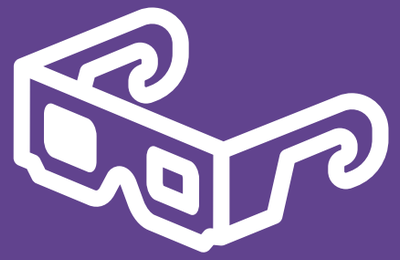


## 2. Instrucciones específicas o personalizadas

Usa instrucciones personalizadas o específicas para:

- Diferentes convenciones entre el código de frontend y backend.
- Guías específicas por lenguaje de programación.
- Patrones propios de un marco de trabajo para módulos concretos.
- Reglas especializadas para archivos de pruebas o documentación.

| PRINCIPALES CABECERAS | | |
|---|---|---|
| Campo | Requerido | Descripción |
| name | NO | Nombre para mostrar en la interfaz. Si no se especifica, se usa el nombre del archivo. |
| description | NO | Descripción breve que aparece al pasar el cursor en la vista de chat. |
| applyTo | NO | Patrón *glob* que define a qué archivos se aplican automáticamente las instrucciones, relativo a la raíz del workspace. Usar **** para aplicarlo a todos los archivos. Si no se especifica, las instrucciones no se aplican automáticamente, aunque se pueden añadir manualmente a una solicitud de chat. |



# DEMO 2





## 3. Agentes personalizados

Permiten configurar distintas variaciones agénticas adaptadas a roles y tareas específicas de desarrollo. Por ejemplo, se puede crear agentes para un revisor de seguridad, un planificador, un arquitecto de soluciones u otros roles especializados. Cada personalidad puede tener su propio comportamiento, herramientas disponibles, modelo de IA asociado e instrucciones.

Son archivos con la extensión `.agent.md`.

También permite definir delegaciones entre agentes (*handoffs*), para crear flujos de trabajo (*workflows*). Por ejemplo, pasar de un agente de planificación directamente a uno de implementación, o transferir el contexto relevante a un revisor de código.

Los agentes pueden utilizar herramientas (*tools*) habitualmente disponibles a través de extensiones o instancias de MCP (Model Context Protocol).

# 3. Agentes personalizados



| PRINCIPALES CABECERAS | | |
|---|---|---|
| Campo | Requerido | Descripción |
| description | NO | Descripción breve del agente. |
| name | NO | Nombre del agente. Si no se especifica, se usa el nombre del archivo. |
| argument-hint | NO | Texto mostrado en el campo de entrada del chat para guiar a los usuarios sobre cómo interactuar con el agente. |
| tools | NO | Lista de nombres de herramientas o conjuntos de herramientas disponibles para este agente. Puede incluir herramientas integradas, conjuntos de herramientas, herramientas MCP o herramientas aportadas por extensiones. Si no se establece, se usan las herramientas establecidas o seleccionadas al momento de invocar. |
| agents | NO | Lista de nombres de agentes disponibles como subagentes. Usar **`*`** para permitir todos los agentes, o un *array* vacío (**`[]`**) ara impedir su uso. |
| model | NO | Modelo de IA que se utilizará al ejecutar el agente. Puede ser un nombre de modelo o una lista priorizada (*array*). Si se especifica un *array*, el sistema prueba cada modelo en orden hasta encontrar uno disponible. Si no se especifica, se usa el modelo seleccionado actualmente. |
| user-invocable | NO | Indicador para controlar si el agente aparece en el menú desplegable de agentes en el chat (por defecto es **`true`**). Ponerlo en **`false`** para crear agentes accesibles solo como subagentes. |
| disable-model-invocation | NO | Indicador para evitar que el agente sea invocado como subagente por otros agentes (por defecto es **`false`**). |



## 3. Agentes personalizados

Ejemplo de delegación (*handoffs*) en agentes personalizados:

```
---
description: Generate an implementation plan
tools: ['search', 'fetch']
handoffs:
  - label: Start Implementation
    agent: implementation
    prompt: Now implement the plan outlined above.
    send: false
    model: GPT-5.2 (copilot)
---
```



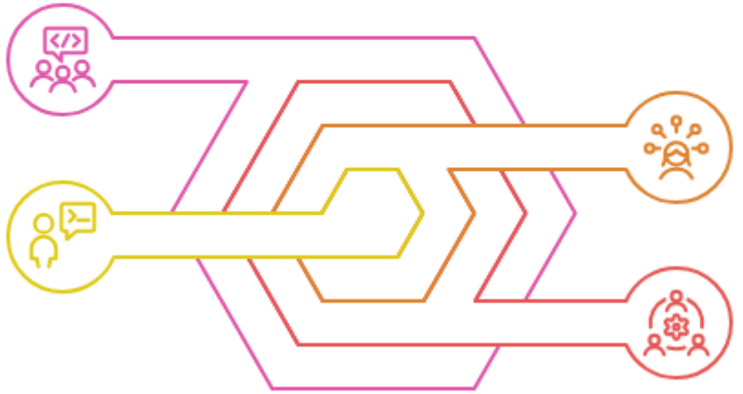
# Agentes personalizados de GitHub Copilot

## Agentes especializados

Arquitecto de software, diseñador de interfaces de usuario, implementador de React, revisor de código

## Agente personalizado

Archivo Markdown con extensión .agent.md



## Función y competencias

Su función, sus competencias y sus herramientas

## Colaboración

Cómo colaborar con otros agentes



## Handoff entre agentes

### Continuidad del trabajo

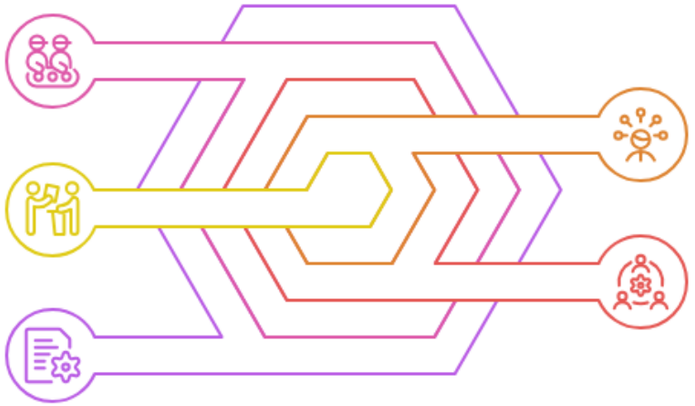
el trabajo puede continuar con los siguientes agentes

### Transferencia de trabajo

de un agente a otro

### Configuración necesaria

mecanismos de delegación, las instrucciones y los permisos necesarios



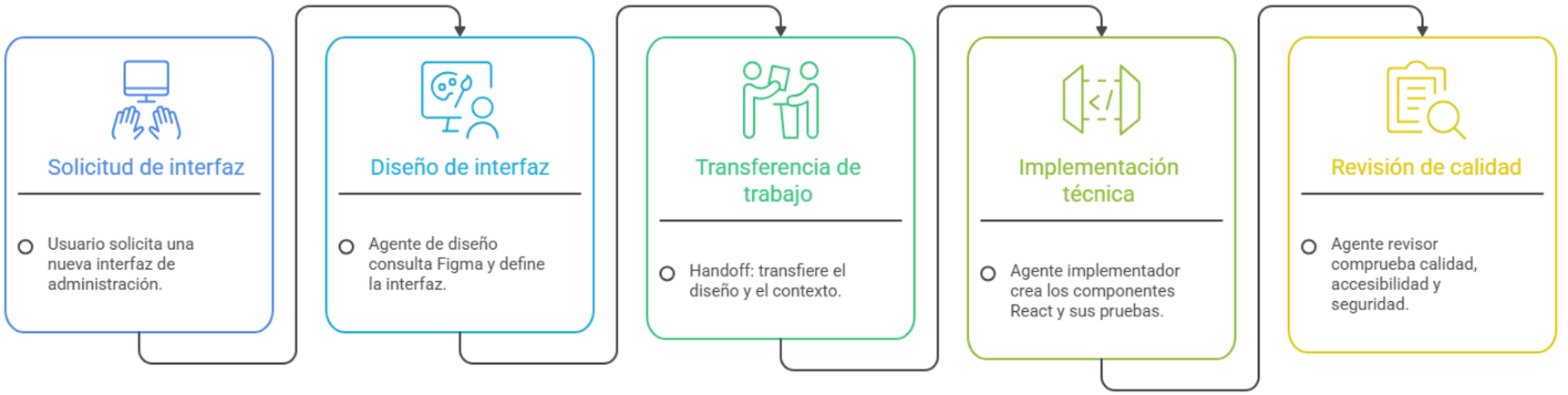
### Especialización

cada agente se ocupa de la parte para la que está especializado

### Flujo de ejemplo

Agente de diseño; Agente implementador; Agente revisor

## Proceso de handoff en flujos de trabajo



# DEMO 3



encamina



## 4. Prompts específicos o personalizados

Permiten guardar prompts para tareas repetitivas o constantes que se quieran o necesiten ejecutar con cierta recurrencia. No se incluyen de forma automática como parte del contexto de GHC, sino que deben ser agregados o invocados explícitamente. Son archivos con la extensión `.prompts.md`.

También se les conoce con el término de «*slash commands*» ya que se invocan usando el `/`

Se recomienda su utilización para:

- Tareas comunes, como generar la estructura de un nuevo componente, ejecutar y corregir pruebas, o preparar una *pull request*.
- Sobrescribir el comportamiento predeterminado de un agente personalizado (llamar a un agente), por ejemplo, creando un plan de implementación o generando *mockups* para llamadas a APIs.

# 4. Prompts específicos o personalizados



| PRINCIPALES CABECERAS | | |
|---|---|---|
| Campo | Requerido | Descripción |
| description | NO | Una descripción breve del *prompt*. |
| name | NO | El nombre del *prompt*, usado después de escribir **/** en el chat. Si no se especifica, se usa el nombre del archivo. |
| argument-hint | NO | Texto de ayuda mostrado en el campo de entrada del chat para guiar a los usuarios sobre cómo interactuar con el *prompt*. |
| agent | NO | El agente usado para ejecutar el *prompt*: *ask*, *agent*, *plan* o el nombre de un agente personalizado. Por defecto, se usa el agente actual. Si se especifican herramientas, el agente predeterminado es *agent*. |
| model | NO | El modelo de lenguaje utilizado al ejecutar el *prompt*. Si no se especifica, se usa el modelo seleccionado actualmente. |
| tools | NO | Lista de nombres de herramientas o conjuntos de herramientas disponibles para este *prompt*. Puede incluir herramientas integradas, conjuntos de herramientas, herramientas MCP o herramientas aportadas por extensiones. |



# DEMO 4





## 5. Habilidades (*Skills*) de agentes

De las topologías disponibles en GitHub Copilot, esta es la adición más reciente y de las actualmente más utilizadas.

Son directorios que contienen instrucciones, *scripts* y recursos (*assets*) que GHC puede cargar cuando son relevantes para realizar tareas especializadas. Está basado en el «Agent Skills», un estándar abierto de Anthropic que funciona en múltiples asistentes de codificación.

A diferencia de las instrucciones personalizadas, que se centran principalmente en definir guías de codificación, las *skills* habilitan capacidades y flujos de trabajo (*workflows*) especializados.

El archivo principal se conoce como el `SKILL.md`.

## 5. Habilidades (*Skills*) de agentes



Beneficios clave de utilizar *skills*:

- Especialización: adapta las capacidades a tareas específicas de un dominio sin repetir contexto.
- Reducir repetición: crea una vez y úsalo automáticamente en todas las conversaciones.
- Componer capacidades: se pueden combinar múltiples *skills* para construir flujos de trabajo complejos.
- Carga eficiente: solo se carga el contenido relevante en el contexto cuando es necesario.

# 5. Habilidades (*Skills*) de agentes

| Función | Agent Skills | Instrucciones Personalizadas |
|---|---|---|
| Propósito | Enseñar capacidades y flujos de trabajo especializados | Definir estándares y guías de codificación |
| Contenido | Instrucciones, *scripts*, ejemplos y recursos | Solo instrucciones |
| Alcance | Específico por tarea, se carga bajo demanda | Siempre aplicadas o mediante patrones *glob* |



Usar *skills* para:

- Crear capacidades reutilizables que funcionen en distintas herramientas de IA.
- Incluir *scripts* y recursos (*assets*) junto con las instrucciones.
- Definir flujos de trabajo especializados, como pruebas, depuración o procesos de despliegue.

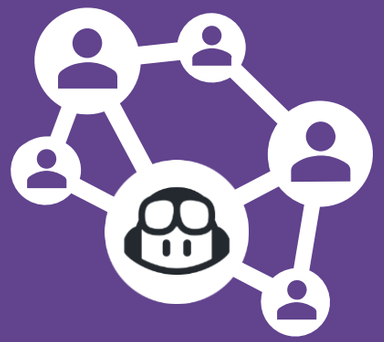
Usar instrucciones personalizadas para:

- Definir estándares de codificación específicos (*ad-hoc*) de un proyecto.
- Establecer convenciones de lenguaje o marco de trabajo (*framework*).
- Especificar pautas para revisiones de código o mensajes de *commits*.
- Aplicar reglas basadas en tipos de archivo usando patrones *glob*.

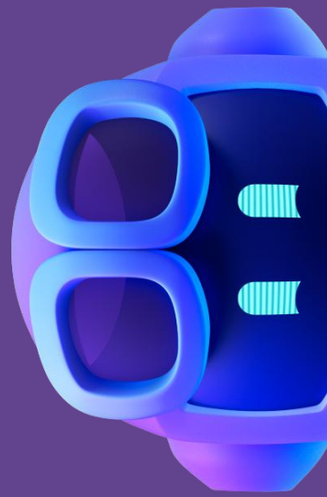
# 5. Habilidades (*Skills*)



| PRINCIPALES CABECERAS | | |
|---|---|---|
| Campo | Requerido | Descripción |
| name | SI | Un identificador único para la skill. Debe estar en minúsculas y usar guiones para los espacios. Debe coincidir con el nombre del directorio padre. Máximo 64 caracteres.<br><br>Ejemplo: `webapp-testing` |
| description | SI | Una descripción de lo que hace la *skill* y cuándo usarla. Se recomienda ser específico, concreto, muy descriptivo y nada ambiguo sobre las capacidades y los casos de uso para ayudar a GHC a decidir cuándo cargar la *skill*. Máximo 1024 caracteres. |
| argument-hint | NO | Texto de ayuda que se muestra en el campo de entrada del chat cuando la *skill* se invoca como comando **/**. |
| user-invocable | NO | Controla si la *skill* aparece como comando **/** en el menú del chat. El valor predeterminado es **`true`**. Al establecerlo en **`false`** se oculta la *skill* del menú, aunque se sigue permitiendo que GHC la cargue automáticamente. |
| disable-model-invocation | NO | Controla si un agente puede cargar automáticamente la *skill* según su relevancia. El valor predeterminado es **`false`**. Se debe establecer en **`true`** para requerir invocación manual mediante el comando **/** únicamente. |



# DEMO 5





## 6. Archivo para agentes (`AGENT.MD`)

Es un único archivo, habitualmente a nivel raíz del proyecto. Se le suele denominar el `README.md` para los agentes de IA, y se pueden tener varios anidados en diferentes directorios.

Los `README.md` están pensados para humanos: guías de inicio rápido, descripciones del proyecto y pautas de contribución. Aunque una IA también lo puede leer, lo cierto es que suelen necesitar más contexto adicional (más detalles). El AGENT.md puede especificar cosas tales como: pasos de compilación, pruebas y convenciones que podrían saturar otra documentación o no ser relevantes para quienes contribuyen como humanos.

Básicamente contiene lineamientos generales para que un agente de IA pueda entender holísticamente el proyecto (o repositorio), su propósito, como construirlo, probarlo o trabajar con el mismo.

En el contexto de GitHub Copilot, a veces, suele ser tan simple como redirigir al `copilot-instructions.md`, o viceversa.

# 6. Archivo para agentes (`AGENT.MD`)



| | `copilot-instructions.md` | `AGENT.md` |
|---|---|---|
| Audiencia | Exclusivo de GitHub Copilot (GHC). | Genérico o más universal. Un agente autónomo de programación, por ejemplo, un agente de codificación de o flujos de trabajo agénticos. En GHC, complementa al `copilot-instructions.md`. |
| Propósito | Orientación específica para GHC: estándares de codificación, bibliotecas o APIs preferidas, patrones, nomenclatura, formato, expectativas de pruebas y, a veces, fragmentos de *prompts* para contribuyentes. | Indicar al agente cómo operar dentro del repositorio: cómo explorar el código, qué convenciones seguir, cómo ejecutar pruebas, cómo estructurar *commits* o *pull requests*, qué evitar, qué reforzar, etc. |
| Usando para... | Uso constante e interactivo de GHC (prompts en chat, autocompletado) para guiar las sugerencias dentro de este repositorio. También es utilizado por otros modos como el modo agente. | Normalmente cuando el agente realiza trabajo de varios pasos (planificación, búsqueda en el repositorio, edición de archivos, ejecución de comprobaciones) y necesita procedimientos operativos específicos del repositorio. |
| Alcance | Suele ser más orientado al estilo y convenciones de código (qué y cómo generar) y un poco menos a los pasos de flujo de trabajo autónomo (cómo ejecutar). | Suele ser más orientado al proceso (flujos de trabajo, herramientas, listas de verificación, límites) que la documentación de producto. |
| Regla práctica | Para orientar el código que genera GHC y su guía durante el desarrollo normal. | Para controlar el comportamiento del agente o su ejecución de extremo a extremo. |

# TRUCOS PARA CREAR INSTRUCCIONES PERSONALIZADAS

- Mantener los *prompts* cortos e independientes. Cada *prompt* debe ser una sola declaración simple.
- Si se necesita proporcionar varios datos, utiliza varias instrucciones separadas o segregadas.
- En la medida de lo posible, evitar hacer referencia a recursos externos en las instrucciones, como estándares de codificación específicos, a menos que se utilice herramientas para navegar o realizar búsquedas por Internet.
- Dividir las instrucciones en varios archivos. Este enfoque es útil para organizar las instrucciones por tema, tecnología, función, propósito o área de conocimiento.
- Facilitar el intercambio de instrucciones personalizadas entre equipos o entre proyectos almacenando las instrucciones en archivos bajo control de versiones y configuración, para poder así garantizar seguimiento de los cambios y evoluciones a lo largo del tiempo.
- Utilizar la propiedad `applyTo` en el encabezado del archivo de instrucciones para aplicar automáticamente las instrucciones a archivos o carpetas específicos.
- Organizar las instrucciones personalizadas en los archivos de solicitudes para mantenerlas limpias y enfocadas, y para evitar duplicar instrucciones para diferentes tareas.
- Dependiendo de la organización y del proyecto, puede que algunos archivos deben ser excluidos del control de versiones o protegidos de ser modificados.

UNA BUENA REFERENCIA

↓

UNA BUENA INSPIRACIÓN

🔗 https://github.com/github/awesome-copilot

# 🤖 Awesome GitHub Copilot Customizations

Powered by Awesome Copilot

all contributors 86

A curated collection of prompts, instructions, and chat modes to supercharge your GitHub Copilot experience across different domains, languages, and use cases.

## 🚀 What is Awesome GitHub Copilot?

This repository provides a comprehensive toolkit for enhancing GitHub Copilot with specialized:

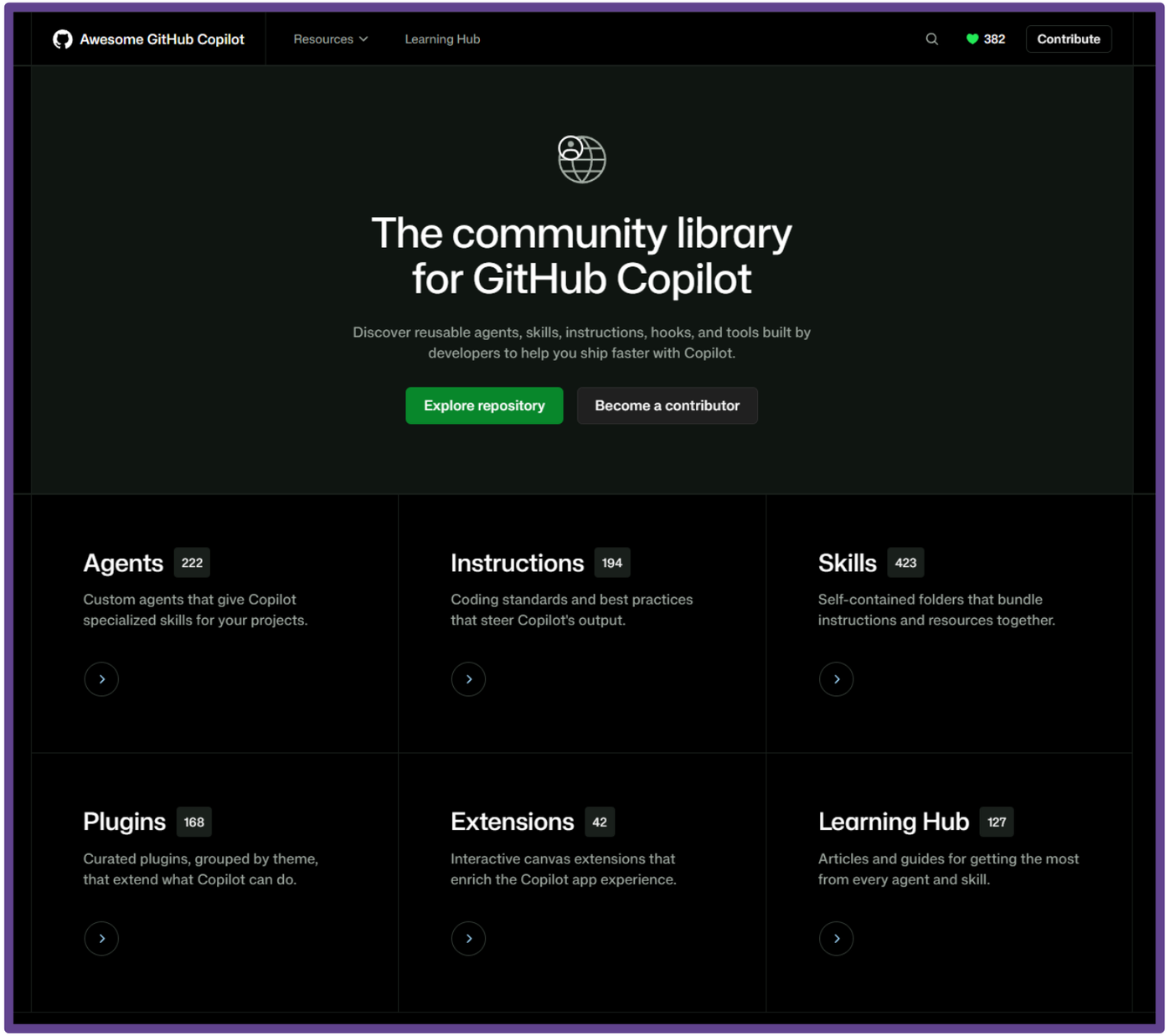
- Awesome Prompts - Focused, task-specific prompts for generating code, documentation, and solving specific problems
- Awesome Instructions - Comprehensive coding standards and best practices that apply to specific file patterns or entire projects
- Awesome Chat Modes - Specialized AI personas and conversation modes for different roles and contexts
- Awesome Collections - Curated collections of related prompts, instructions, and chat modes organized around specific themes and workflows

UNA BUENA REFERENCIA

↓

UNA BUENA INSPIRACIÓN

🔗 https://awesome-copilot.github.com/




encamina



# Políticas de prompts

Son las directrices y reglas institucionalizadas por la organización para el uso adecuado de la IA y la redacción de *prompts*.

Incluyen lineamientos y recomendaciones sobre cómo escribir *prompts* claros, seguros y efectivos, que sigan las prácticas de desarrollo de la empresa y sus normas de seguridad.

Buscan que el uso de la IA sea responsable, correcto, ético, y alineado con los estándares de calidad y seguridad de la organización.




# Políticas de prompts

Por ejemplo, una política típica es evitar lenguaje vago o ambiguo en las solicitudes y formular indicaciones lo más precisas posible.

También se establecen restricciones para garantizar la confidencialidad, como no incluir datos sensibles o código propietario en *prompts* que puedan ser procesados fuera del entorno seguro, o promover la revisión humana de las sugerencias de antes de incorporarlas al código productivo.

# RECOMENDACIONES PARA POLÍTICAS DE PROMPTS

- Definir el lenguaje en el cual se escribirán los archivos. Por ejemplo: español, inglés, etc. Lo ideal es elegir y mantener un único lenguaje. En ENCAMINA solemos recomendar el inglés.
- Definir un estándar y catálogo de nombres para los archivos de agentes, habilidades (*skills*), instrucciones, etc.
- Establecer el procedimiento de conservación y control de versiones de los archivos.
- Definir y establecer el conjunto de extensiones y herramientas (MCPs) que deben ser utilizados como parte de la dinámica de trabajo, así como los lineamientos que permitan identificar cuándo usarlos y cuando no.

# RECOMENDACIONES PARA POLÍTICAS DE PROMPTS

- Establecer un comité o grupo privilegiado de revisores que se encarguen de constante y continuamente auditar la calidad de los archivos y el estricto cumplimiento de las políticas que se establezcan.
- Definir que cabeceras de los archivos, aun siendo opcionales, son políticamente consideradas obligatorias.
- Definir y establecer que modelos de IA (LLMs) a usar para cada casuística, con recomendaciones generales, más no necesariamente estrictas.
- Establecer los procedimientos y reglas para definir delegaciones (*handoffs*) o incluso no permitirlas si así se establece a nivel cultural dentro de la organización.
- Mantener estas políticas lo más simple y sencillo posible, pues en caso contrario existe un alto riesgo de incumplimiento.

# APM

## Agent Package Manager

Es un gestor de dependencias para activos y artefactos empleados por asistentes de programación con Inteligencia Artificial.

Declara las *skills*, *instrucciones*, *plugins* y servidores MCP que necesita un proyecto en un único archivo (`apm.yml`), y luego cualquier persona ejecuta `apm install` para desplegar cada activo o artefacto en los *harnesses* que la soporten.

Permite definir políticas de seguridad y de cumplimiento para que se utilicen activos y artefactos certificados o validadoes por la organización.

Los componentes gestionados por esta herramienta están disponibles en *maketplaces* o catálogos tanto públicos, y también soporta el uso de instancias privadas, con lo cual una organización podrá contar con su propio conjunto de artefactos gestionados con esta herramienta.

encamina

# APM

## Agent Package Manager

https://microsoft.github.io/apm/

# Agent Package Manager

A dependency manager for AI agents -- like npm for agent context.

Quickstart → View on GitHub

APM is a dependency manager for AI agents. Declare the skills, prompts, instructions, plugins, and MCP servers your project needs in one `apm.yml`, then any developer runs `apm install` to deploy each primitive to the harnesses that support it. See the targets matrix for support across GitHub Copilot, Claude Code, Grok Build, Cursor, OpenCode, Codex, Gemini, Windsurf, and Kiro.

### Portable by manifest

One `apm.yml` across supported harnesses and machines. The lockfile pins exact versions and content hashes so a fresh clone resolves the same package content byte-for-byte.

### Secure by default

Every install scans for hidden Unicode, pins content hashes, and blocks transitive MCP servers unless they are explicitly declared or trusted. No opt-in required.

### Governed by policy

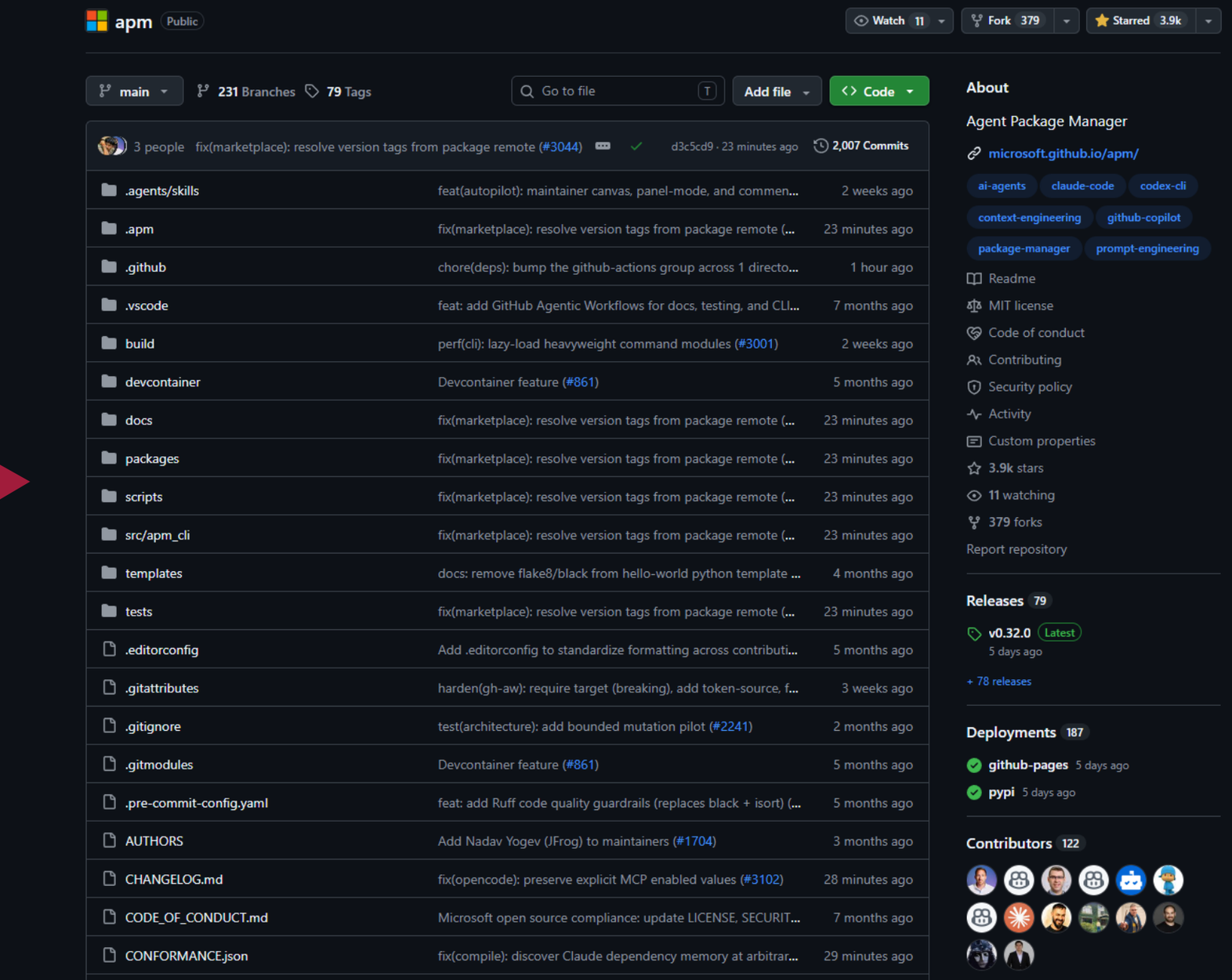
`apm-policy.yml` is enforced at install time, including transitive MCP. Tighten-only inheritance flows enterprise -> org -> repo. Runtime behavior is your harness's domain. See the Governance Guide.

encamina

# APM

## Agent Package Manager

https://github.com/microsoft/apm




encamina

# Del archivo suelto al paquete reproducible

### apm.yml

Declara agentes, skills, prompts, instrucciones y servidores MCP.

### apm.lock.yaml

Fija versiones, commits y hashes de contenido para repetir la instalación.

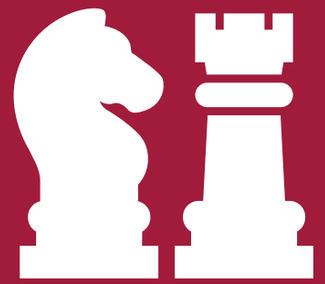
### apm install

Distribuye cada pieza a los entornos compatibles con esa topología.

**Un manifiesto compartido. Contexto consistente en cada proyecto.**



# DEMO 7



# ¡Gracias! Para localizar o contactar con ENCAMINA puedes:

Enviar un mail a:

info@encamina.com
administracion@encamina.com

O llamar al:

Madrid +34 917 893 823
Valencia +34 962 698 064

O enviar un fax al 962 698 063

O hablar personalmente con tu Account Manager

O visitarnos en:

C/O'Donnell, 34
28009, **Madrid**

---

C/Fundició, 79
46520 Port de Sagunt, **València**

---

InnovaParq Universidad de La Laguna
Av. Trinidad, 61. Campus Central
38200, S. Cristóbal de La Laguna, **Tenerife**

Puedes encontrarnos en:

